Hackathon Project Document

AI Secure Data Intelligence Platform

(AI Gateway + Scanner + Log Analyzer + Risk Engine)

1. Overview

The **AI Secure Data Intelligence Platform** is an advanced system designed to act as:

- **AI Gateway**
- **Data Scanner**
- **Log Analyzer**
- **Risk Engine**

Key Capability Expansion

In addition to processing structured and unstructured data, the system now:

- Ingests and analyzes **log files**
- Detects **security risks, secrets, and anomalies**
- Generates **AI-powered insights from logs**

2. Objective

The platform must support **multi-source data ingestion and intelligent analysis**.

Supported Input Types

- Text input
- Files:
    - PDF
    - DOC
    - TXT

- Log files
- SQL / structured data
- Live chat input

3. Log File Processing

3.1 Supported Formats

- `.log`
- `.txt`

3.2 Log Characteristics

Log files may contain:

- System events
- API responses
- Errors
- Debug traces
- Sensitive data leaks

3.3 Log Analysis Requirements

The system **must perform the following**:

Sensitive Data Detection

Identify and extract:

- Emails
- Phone numbers
- API keys
- Passwords
- Tokens

Security Issue Detection (CRITICAL)

Detect:

- Hardcoded secrets
- Credentials in logs
- Suspicious patterns
- Error leaks (e.g., stack traces)

AI-Based Log Insights

Generate:

- Summary of log activity
- Detected anomalies
- Potential risks

Example Insights

- “API key exposed in logs”
- “Multiple failed login attempts detected”
- “Sensitive user data logged in plain text”

4. Processing Engine

4.1 Log-Specific Detection Layer

Introduce a **Log Analyzer Module**:

```
Log Input
  ↓
Parsing (line-by-line / chunking)
  ↓
```

```
Pattern Detection
  ↓
AI Analysis
  ↓
Risk Classification
```

4.2 Log Risk Patterns

Pattern	Risk Level
API key in logs	High
Password in logs	Critical
Email in logs	Low
Stack trace	Medium

4.3 Optional Advanced Detection

- Repeated failures (brute-force detection)
- Suspicious IP activity
- Debug mode leaks

5. End-to-End Processing Flow

```
Input (Text / File / SQL / Log / Chat)
        ↓
Validation
        ↓
Extraction (Parser)
        ↓
Detection Engine
   ├── Regex
   ├── AI
   ├── Log Analyzer (NEW)
        ↓
Risk Engine
```

```
        ↓
Policy Engine
        ↓
Response
```

6. API Design

Endpoint

```
POST /analyze
```

Request Format

```
{
  "input_type": "text | file | sql | chat | log",
  "content": "...",
  "options": {
    "mask": true,
    "block_high_risk": true,
    "log_analysis": true
  }
}
```

Response Format

```
{
  "summary": "Log contains sensitive credentials and errors",
  "content_type": "logs",
  "findings": [
    {
      "type": "api_key",
      "risk": "high",
      "line": 12
    },
    {
      "type": "password",
```

```
      "risk": "critical",
      "line": 25
    }
  ],
  "risk_score": 10,
  "risk_level": "high",
  "action": "masked",
  "insights": [
    "Sensitive credentials exposed in logs",
    "Debug information leaked"
  ]
}
```

7. Frontend Requirements

Log Upload UI

- Upload support for:
  - `.log`
  - `.txt`
- Drag & drop *(Bonus)*

Log Visualization (BONUS)

Display logs with:

- Highlighted sensitive lines
- Line numbers
- Risk markers

Insights Panel (NEW)

Display:

- AI-generated log summary

- Security warnings
- Risk breakdown

8. Evaluation Criteria

Category	Marks
Backend Design	18
AI Integration	15
Multi-Input Handling	12
Log Analysis (NEW)	15
Detection + Risk Engine	12
Policy Engine	8
Frontend UI	10
Security	5
Observability	3
Bonus	2

9. Example Use Case (Log File Analysis)

Input: `app.log`

```
2026-03-10 10:00:01 INFO User login
email=admin@company.com
password=admin123
api_key=sk-prod-xyz
ERROR stack trace: NullPointerException at service.java:45
```

Expected Output

```
{
  "summary": "Log contains sensitive credentials and system errors",
  "findings": [
    {
      "type": "email",
```

```
      "value": "admin@company.com",
      "risk": "low"
    },
    {
      "type": "password",
      "risk": "critical"
    },
    {
      "type": "api_key",
      "risk": "high"
    }
  ],
  "risk_score": 12,
  "risk_level": "high",
  "insights": [
    "Sensitive credentials exposed",
    "Stack trace reveals internal system details"
  ]
}
```

10. Advanced Challenges

Participants can attempt advanced implementations such as:

- Real-time log streaming analysis
- Cross-log anomaly detection
- Correlation across multiple log entries
- Rate limiting for large log uploads
- Efficient chunking for large log files

Final Notes

- Log analysis is **mandatory** and a **high-weight evaluation criterion**
- System should be **modular, scalable, and secure**
- AI insights should be **meaningful, not generic**